

PoTranslator

Version 2.0

Multi-language localization workspace for gettext PO catalogs

USER MANUAL

Created by Daniel Perdomo
Ultraton Studios

Contents

1. What PoTranslator Does	2
2. Getting Started	3
3. The Workspace	5
4. Change States and Colors	7
5. Context: The Three Layers	8
6. Fitting Text Into the Interface	10
7. Updating the Base Catalog	11
8. Translating	12
9. Reviewing and Correcting	14
10. Exporting	15
11. Reference	16
12. Troubleshooting	17

CHAPTER 1

What PoTranslator Does

PoTranslator turns a gettext **PO** or **POT** catalog into finished translations for up to nineteen target languages, using OpenAI or Claude models. It is built for software and video-game localization, where the same short string appears hundreds of times, where a misplaced `{variable}` breaks the build, and where a button label that grows by 40% breaks the layout.

It is a desktop application for Windows. There is nothing to install: a single executable holds the whole program.

Why a dedicated tool

Pasting strings into a chat window loses the two things that decide translation quality: **context** and **consistency**. PoTranslator keeps both in one project file:

- **Context travels with the request.** Three layers of instruction reach the model on every batch: what the product is, how a specific language must word things, and what an individual line means on screen.
- **Work is incremental.** Update the base catalog and the tool tells you exactly what is new, what changed, and what disappeared. Existing translations survive.
- **Every change is tracked per language.** Correcting Spanish never marks Portuguese as needing attention.
- **Placeholders are validated.** If the model drops a `%d` or renames a `{player}`, the entry is flagged instead of silently shipped.
- **Duplicates are paid for once.** Identical source strings with identical context are sent to the model a single time and the answer is copied to every occurrence.

What you need

An API key from OpenAI or Anthropic (Claude). The key is yours; PoTranslator never ships one and never sends your catalog anywhere except to the provider you choose.

CHAPTER 2

Getting Started

Step 1 · Store an API key

Open **Settings** in the menu bar and choose **Configure OpenAI API key** or **Configure Claude API key**. Paste the key and confirm.

The key is encrypted with Windows DPAPI and tied to your Windows user account. It is stored outside the project, at:

```
%LOCALAPPDATA%\Ultraton\PoTranslator\credentials.json
```

Nobody else signed into the machine can decrypt it, and it is never written into a .potranslator file, so projects are safe to share.

ENVIRONMENT VARIABLES TAKE PRIORITY

If OPENAI_API_KEY, ANTHROPIC_API_KEY or CLAUDE_API_KEY is set in your environment, that value is used and the stored key is ignored. Useful on a build machine; surprising if you forgot one was set.

Step 2 · Create a project

The fastest route is **Import base** on the toolbar. Pick your source .po or .pot file, then pick a folder to keep the project in. PoTranslator creates two things there:

- **<name>.potranslator** — the project file. It holds every source string, every translation, all your context and settings, and the change history.
- **PO_Files** — the folder every export is written into, one .po per language.

You can also use **+ New** to create an empty project first and import the base afterwards.

Step 3 · Set the source language

SOURCE LANGUAGE in the left panel tells the model what it is translating *from*. Leave it on **<AUTO>** and the language is read from the PO header. Setting it explicitly is safer when the header is missing or wrong.

Whatever you choose disappears from the target list below, because translating a catalog into its own language is never wanted. Choose Spanish as the source and Spanish vanishes while English appears.

Step 4 · Choose target languages

Tick the languages you want in the **Selected** column of the **LANGUAGES** list. Those ticks drive both **Translate selected languages** and **Export selected languages**.

Step 5 · Translate

In the right panel, set **MESSAGE SCOPE** to **New messages** and press **Translate selected languages**. Progress appears in the status bar at the bottom of the window.

Step 6 · Export

Press **Export selected languages**. One .po per language lands in the PO_Files folder, named after its language code (es.po, pt.po, and so on).

CHAPTER 3

The Workspace

The window is divided into a toolbar, three panels, and a status bar. The divider between panels can be dragged.

Toolbar

Button	What it does
+ New	Create an empty project in a folder you choose.
Open	Open an existing .potranslator file.
Save	Write the project to disk. Ctrl+S. The project also autosaves 1.5 seconds after any edit.
Import base	Import or update the source catalog. Safe to run repeatedly — see Chapter 7.
Import translation	Load an existing translated PO into one language, so previous work is not redone.
Export selected languages	Write a PO for every ticked language.
Export all languages	Write a PO for every language except the source.

Left panel · project settings

Everything here is stored inside the project and applies to the whole catalog.

Control	Purpose
SOURCE LANGUAGE	The language you are translating from. Hidden from the target list.
GENERAL CONTEXT	Background sent with every request, in every language. See Chapter 5.
CONTEXT · <LANGUAGE>	Wording rules for one language only. The heading follows the language selected in the right panel.
PROVIDER	OpenAI or Claude. Each keeps its own stored key.
MODEL	Which model to use. The list changes with the provider.
PROFILE	Economy, Balanced, or Maximum quality. See Chapter 9.
BATCH	Messages per request, 1 to 100. Default 30.
UI WORDS	The short-string brevity rule. Default 4, and 0 disables it. See Chapter 6.
LANGUAGES	The target list, with a tick box per language.

HOVER FOR HELP

GENERAL CONTEXT, the per-language context box, and UI WORDS all show an explanatory tooltip when you rest the mouse on their label or field for half a second.

Center panel · the message list

One row per source string. The **Translation** column and the **Change** and **Status** columns always reflect the language chosen in the right panel, named in the column header.

- **Search box** — matches source text, context, tags and the translation itself.
- **Status filter** — All, New, Modified, Pending, Review, Errors, Ready.
- **Tag filter** — the narrow box to the right of the status filter.
- **Range** — tick a numbered span in one action, then **Check range**.
- **All / None** — tick or clear every currently visible row.
- **Mark for reprocessing** — set the ticked rows back to Pending in the ticked languages.
- **Tag** — attach a label to the ticked rows for later filtering.

Click the circle in the first column to tick a row. Click anywhere else to open it in the details panel. Double-click translates that single row immediately.

Right panel · details and translation

The upper half works on the selected line; the lower half runs bulk translation.

- **SOURCE TEXT** — read-only original, plus the plural form when there is one.
- **LINE CONTEXT** — your note about what this string means on screen. Shared by all languages.
- **TAGS** — comma-separated labels for filtering.
- **TRANSLATION** — the editable translation, with the language picker beside it. Changing that picker re-aims the whole message list.
- **PLURALS** — one `index=translation` per line.
- **Save manual correction** — store your edit, the line context and the tags.
- **Reprocess this line** — re-translate just this row.
- **Copy source to translation** — for strings that must not change, such as product names.
- **MESSAGE SCOPE** and the two Translate buttons — see Chapter 9.

Status bar

Along the bottom edge: the current activity on the left, and on the right a running count, a percentage, and a progress bar. A **Cancel** button appears there while a translation is running.

CHAPTER 4

Change States and Colors

Every row carries a **Change** state and a **Status**. They answer different questions, and both are calculated **per language**.

Change · does this need my attention?

State	Row color	Meaning for the selected language
New	Blue	This language has no translation yet.
Modified	Amber	It carries a manual correction of yours, or a translation produced against an older source text.
Unchanged	None	Translated and up to date.
Removed	Grey	The string is gone from the base catalog. Global, not per language.

Because this is per language, fixing a Spanish wording marks the Spanish row amber and leaves Portuguese untouched. Switch the language picker and the colors change with it.

Status · what happened to this entry?

Status	Meaning
Pending	Not translated yet.
Ready	Translated by the model and validated.
Manual	You wrote or edited it by hand. Never overwritten by a bulk run.
Review	Needs a human look: imported as fuzzy, or the source text changed underneath it.
Error	The request failed, the model omitted the entry, or placeholder validation rejected the result. The reason is shown under the translation box.

A RED ROW IS NOT LOST WORK

Errors keep whatever text was produced and stay in the catalog. Set MESSAGE SCOPE to **Pending / review / errors** and run again to retry only those.

CHAPTER 5

Context: The Three Layers

Context is what separates a usable translation from a literal one. PoTranslator sends three layers, from broadest to most specific. When they conflict, the more specific one wins.

1 · General context

Left panel. Sent with every request in every language. Describe the product, the tone, and any term that must never be translated.

```
Arcade football game mixing soccer with air hockey.
Characters are called Roomballs. Informal, energetic tone.
Never translate: GOAL, COMBO, Roomballs, Roombolts.
```

2 · Language context

Left panel, directly below the general context. The heading names the language it belongs to — **CONTEXT · SPANISH** — and follows the language picker in the right panel. Use it for wording rules that apply to one language and no other.

```
kick = "patear", never "chutar".
Address the player as tu, not usted.
Use Latin American vocabulary, not peninsular Spanish.
```

This is the right home for a glossary. If a model keeps choosing a word you dislike in one language, write the rule here rather than in the general context, where it would waste tokens on the other eighteen languages and could confuse them.

3 · Line context

Right panel, per message, shared by all languages. Use it when the source string alone is ambiguous.

- Right — is that a direction or the opposite of wrong? Write *D-pad direction*, *not correctness*.
- Free — no cost, or unoccupied? Say which.
- Standby — a state label on the pause screen.

LINE CONTEXT ALSO SPLITS DUPLICATES

Identical strings are normally translated once and copied. Two rows with the same source but different line context are treated as different strings and translated separately — which is exactly what you want when the same word means two things.

What the model actually receives

Each request carries the instructions, then the project context, then the language rules last so they take priority:

```
You are a senior software and video-game localization translator.
Translate every supplied item from en to Spanish (es).
...
```

```
PROJECT CONTEXT:
Arcade football game. Informal tone.
```

SPANISH LANGUAGE RULES (these override the project context on conflict):
kick = patear, never chutar

CHAPTER 6

Fitting Text Into the Interface

The classic localization failure is a button that works in English and overflows in Spanish. `RESTORE DEFAULTS` is sixteen characters; a literal Spanish rendering reaches thirty-five and pushes the layout apart.

The UI WORDS setting

PoTranslator treats any source string of **N words or fewer** as a user-interface label — a button, tab, menu entry, or title inside a fixed-width control — and instructs the model to keep it terse: a command or noun phrase, never a sentence, dropping articles and prepositions before it exceeds the space.

UI WORDS in the left panel sets N. It ships at **4**, which covers almost every control label in a typical game or application.

Value	Effect
4	Default. Recommended for most interfaces.
Higher	Use when your controls are roomy and longer labels still fit.
Lower	Use when space is very tight, so only the shortest strings are constrained.
0	Switches the rule off entirely. Every string is translated at natural length.

The setting is stored per project, so a website project and a console-game project can use different limits.

LONGER STRINGS ARE UNAFFECTED

A sentence such as "Your progress will be lost if you leave now" is well past the limit and is translated at whatever length reads naturally. The rule only guards short labels.

CHAPTER 7

Updating the Base Catalog

Development moves on: strings are added, reworded, and deleted. Press **Import base** again and pick the new file. This is a **merge**, never a replacement — your translations are not discarded.

What happens to each string

Result	When	Your translations
New	The string did not exist before.	Starts empty, pending in every language.
Modified	The string exists but the text changed.	Kept , and flagged for review in every language that had one.
Unchanged	Identical to before.	Untouched.
Removed	No longer in the base catalog.	Archived, not deleted. Hidden from the list and excluded from exports.

A summary box reports the four counts when the import finishes, and the project is saved immediately.

How a reworded string is recognised

A string is identified by its context, its text, and its plural form together. Change the text and that identity no longer matches, so PoTranslator falls back to a second method: it looks for an **anchor** that survived the edit.

- the `msgctxt` context value;
- a source reference such as `#: ui/menu.cpp:120`, compared without the line number so ordinary code movement does not break the link;
- a comment of the form `key: or id:.`

If exactly one old entry matches an anchor, the two are linked: the translation is carried across and marked for review. If nothing matches, the old string is archived as **Removed** and the new one arrives as **New**.

CHECK YOUR CATALOG FOR ANCHORS

If your PO files contain only `msgid` and `msgstr`, with no context and no `#:` references, there is nothing to anchor to. Editing a string then costs you its translation. Adding `msgctxt` or source references to your extraction step makes every future update far cheaper.

Bringing in translations you already have

Import translation loads an existing translated PO into one language. Entries are matched on context plus text; anything marked **fuzzy** arrives as **Review** rather than **Ready**. The report tells you how many were imported, how many did not match anything in the base, and how many base strings are still without a translation.

CHAPTER 8

Translating

Choosing what to translate

Two controls decide the work: **MESSAGE SCOPE** picks the *messages*, and the button you press picks the *languages*.

Message scope	Selects
New messages	Strings that arrived with the most recent base import.
Checked messages	Only the rows you ticked in the list.
Pending / review / errors	Everything not finished. The natural choice for resuming.
All messages	The entire catalog. Asks for confirmation and shows a token estimate first.

Button	Languages
Translate selected languages	Only the languages ticked in the Selected column.
Translate all languages	Every language except the source.

Work already finished is skipped automatically for the **New** and **Pending / review / errors** scopes, so you can leave every language ticked and simply run again. **Manual** entries are never overwritten by a bulk run.

Quality profiles

Profile	Behavior
Economy	Direct, efficient translations with no stylistic elaboration. Cheapest and fastest.
Balanced	Default. Balances natural wording, consistent terminology, and fit.
Maximum quality	The model verifies meaning, terminology, grammar, placeholders and fit before answering. Slowest and most expensive.

Batch size and cost

BATCH sets how many messages go into one request. Larger batches cost fewer tokens overall, because the instructions and context are sent once per request rather than once per string. The default of 30 suits most catalogs.

The **token estimate** above the scope selector updates as you change settings. It counts the languages you have ticked, and it counts duplicate strings only once. Tick some rows and it narrows to just those.

OVERSIZED BATCHES RECOVER ON THEIR OWN

If a reply is cut off because the batch was too large, PoTranslator splits it in half and retries both halves automatically. You will see a message about splitting in the status bar. Failed requests are retried twice before the entries are marked as errors.

Placeholder validation

Every returned string is checked against its source for formatting tokens: {name}, %d, %(count)s, , \n, \$VAR and similar. If any is missing, added, or altered, the entry becomes an **Error** naming the offending token, and the text is kept so you can repair it by hand.

Cancelling and resuming

Cancel in the status bar stops after the request in flight. Nothing is lost:

- Everything already translated is written into the project and saved automatically.
- The batch in flight finishes and is saved — it is not thrown away mid-request.
- Languages that had not started remain Pending, untouched.

To resume, set the scope to **Pending / review / errors** and run again. There is no need to untick the languages that finished.

ONE SIDE EFFECT OF CANCELLING

If you cancel while a batch is retrying after a network failure, that batch's messages are marked as errors rather than pending. They appear in red. No data is lost and the **Pending / review / errors** scope picks them up, but do not be alarmed by the color.

CHAPTER 9

Reviewing and Correcting

Select a row and the right panel fills in. Edit the **TRANSLATION** box and press **Save manual correction**. The entry becomes **Manual**, which protects it from every future bulk run, and the row turns amber in that language only.

Plurals

Entries with a plural form show a **PLURALS** box, one line per form:

```
0=1 objeto
1={count} objetos
```

The number of forms comes from the language's own plural rule — one for Japanese and Chinese, two for most European languages, three for Russian and Polish, six for Arabic. PoTranslator writes the correct `Plural-Forms` header on export.

Working in bulk

- **Tag** the rows belonging to one screen or feature, then use the tag filter to review them together.
- **Mark for reprocessing** returns ticked rows to Pending so the next run redoes them — useful after you improve the language context.
- **Copy source to translation** for strings that must stay identical, such as product names or key labels.

A practical review loop

- Translate one language first and read through it.
- When you find a recurring wording problem, write the rule into that language's context box.
- Tick the affected rows, press **Mark for reprocessing**, and run again with the scope set to **Pending / review / errors**.
- Once that language reads well, run the rest with confidence.

CHAPTER 10

Exporting

Export selected languages writes the ticked languages; **Export all languages** writes every language except the source. Files are named after the language code and land in the `P0_Files` folder beside the project.

Each exported file carries the original header, updated with the target language, UTF-8 encoding, the correct plural rule, and a generator stamp. Comments, source references and flags from the base catalog are preserved.

- Entries still awaiting review are written with the fuzzy flag, so other gettext tools treat them as provisional.
- Archived (removed) strings are not exported.
- Untranslated entries are written with an empty `msgstr`, which is what gettext expects.

EXPORT OVERWRITES WITHOUT ASKING

A file already present in `P0_Files` for the same language is replaced. If you hand-edited an exported PO, import it back with **Import translation** before exporting again, or your edits are gone.

CHAPTER 11

Reference

The project file

A `.potranslator` file is a ZIP archive holding three items:

Item	Contents
project.json	Every source string, translation, setting, context and history entry.
source.po	The current base catalog, readable by any gettext tool.
format.txt	A format marker.

It contains no credentials, so it is safe to commit to version control or send to a colleague. Saving is atomic: the file is written to a temporary name and swapped into place, so a crash mid-save cannot corrupt the project.

Supported target languages

English, Spanish, Chinese (Simplified), German, French, Portuguese, Russian, Japanese, Korean, Italian, Polish, Turkish, Arabic, Dutch, Thai, Hindi, Vietnamese, Ukrainian, Czech, Indonesian. The source language is hidden from the list.

Keyboard shortcuts

Shortcut	Action
Ctrl+N	New project
Ctrl+O	Open project
Ctrl+S	Save project
Double-click a row	Translate that row now

Settings and their defaults

Setting	Range	Default
Source language	Auto-detect or an explicit language	<AUTO>
Provider	OpenAI or Claude	OpenAI
Quality profile	Economy, Balanced, Maximum quality	Balanced
Batch size	1 to 100 messages	30
UI words	0 to 20 (0 disables the rule)	4

CHAPTER 12

Troubleshooting

The translation stops with an authentication or quota error

The key is wrong, expired, or the account has no credit. Re-enter it under **Settings**. Remember that an environment variable overrides the stored key, so check `OPENAI_API_KEY` and `ANTHROPIC_API_KEY` as well.

Many entries came back as errors

Open one and read the reason under the translation box. A placeholder complaint means the model altered a variable — usually fixed by lowering the batch size or raising the quality profile. A network message means the request never completed; simply run again with the **Pending / review / errors** scope.

A whole batch was truncated

This is handled for you: the batch is split in half and both halves are retried. If it keeps happening, lower `BATCH` to around 15 for catalogs full of long strings.

I updated the base and lost translations

The strings were reworded and there was no anchor to link old to new, so they were treated as new arrivals. Nothing was deleted — the old entries are archived inside the project. Adding `msgctxt` or `#:` references to your catalog prevents this in future. See Chapter 7.

Buttons still overflow in some language

Lower **UI WORDS**, or add an explicit instruction in that language's context box, such as *Menu labels must never exceed two words*. Then tick the offending rows, press **Mark for reprocessing**, and run again.

A word is translated the same wrong way everywhere

Write the rule in the language context box rather than correcting every row by hand, then reprocess the affected rows. The rule applies to that language only and outranks the general context.

A row shows as Modified but I never edited it

Either the source text changed on the last base import, or the translation was imported carrying a fuzzy flag. Both mean the same thing: a human should confirm it. Saving it, or reprocessing it, clears the mark for that language.

Where did my API key go?

It is encrypted for your Windows account under `%LOCALAPPDATA%\Ultraton\PoTranslator`. It cannot be recovered on another machine or by another user, and it is deliberately absent from the project file. Re-enter it under **Settings**.

Can I share a project with a translator?

Yes. Send the `.potranslator` file. It carries every string, translation, context note and setting, and no credentials. They will need their own API key to run translations, but they can review and correct without one.